

0.4 Tensors, Matrices & PyTorch

Introduction: What's Been Hiding in Plain Sight?

Throughout this course, you've already seen neural networks (where you write things like `weights * inputs + bias`) and transformers (where you write `Q = X * Wq`). But what are all of these operations *really* doing under the hood?

The answer is surprisingly simple: **Matrix Multiplication. That's the secret.** Every major operation in modern AI — forward passes, backpropagation, attention — boils down to multiplying grids of numbers together. This lecture breaks that down from scratch.

What this lecture covers:

Section	Topic	Key Question
Part 1	Scalars → Vectors → Matrices → Tensors	What are these things? Why do they matter?
Part 2	Matrix Magic (Transformations)	What can matrices actually <i>do</i> ?
Part 3	PyTorch	The tool that makes it all fast and automatic

PART 1: THE BUILDING BLOCKS

Think of this section as understanding different-sized containers for data. Each one builds on the previous.

1.1 — Scalar: The Simplest Thing

A scalar is just **a single number**. That's it. No list, no grid — just one value.

Now let me build the diagrams to accompany the notes.

Real-World Examples:

- temperature = 72 (degrees)
- user_age = 25 (years)
- coffee_price = 4.99 (rupees)

Dimensions: 0 — a scalar is just a single point in space. No direction, no length.

Simple analogy: Think of a scalar like a single dot on a blank page. It tells you *how much* of something there is, but nothing about direction or structure.

1.2 — Vector: Numbers with Direction

A vector is **a list (array) of numbers**. Instead of one value, you now have multiple values grouped together. Each number in the list represents a different measurement or feature.

Real-World Examples:

- `location = [28.6139, 77.2090]` — latitude and longitude (2 numbers = a point on a map)
- `rgb_color = [255, 87, 51]` — red, green, blue values (3 numbers = a color)
- `movement = [3, 4]` — 3 units right, 4 units up (2 numbers = a direction)

Dimensions: 1 — a vector is a single row (or column) of numbers.

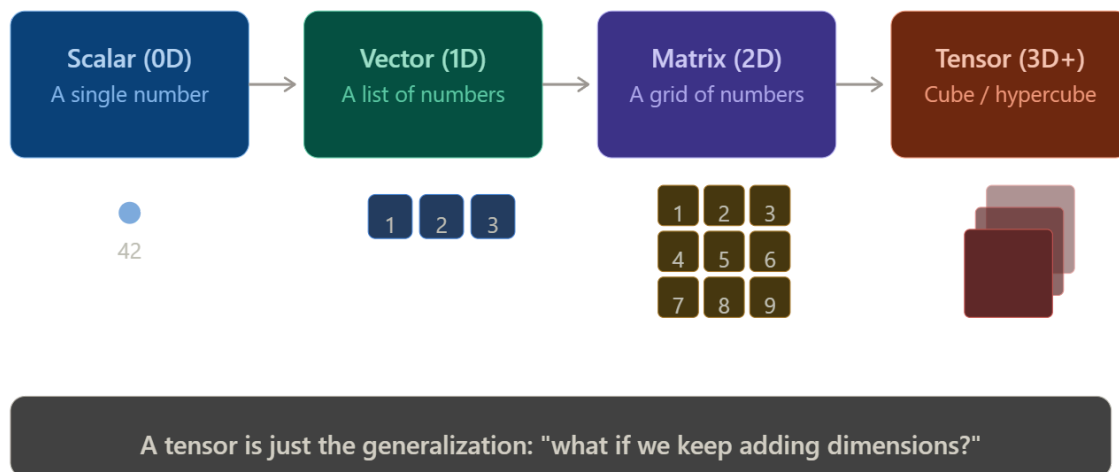
Where you've already seen vectors in AI:

- **Word Embeddings:** The word "King" gets converted into a vector like `[0.2, -0.5, 0.8, 0.1, ...]` with 768 numbers! Each number captures some aspect of meaning. This is how AI "understands" words — by placing them as points in a high-dimensional space.
- **Neural Network Inputs:** When you feed features `[x1, x2, x3]` into a neuron, that's a vector being processed.

Key Insight: These were vectors all along. Every time AI processes a word or a set of features, it's working with vectors.

1.3 — Matrix: A Grid of Numbers

A matrix is **a 2D grid of numbers** arranged in rows and columns. Think of it like a spreadsheet or a table.



Scalar = 0D tensor (a point)

Vector = 1D tensor (a line of numbers)

Matrix = 2D tensor (a table of numbers)

Tensor = 3D, 4D, 5D... (stacked tables, cubes, hypercubes)

Properties of a matrix:

- Two-dimensional structure (rows and columns)
- Indexed by (row, column) position — like a cell in a spreadsheet
- Example: a 3×3 matrix has 3 rows and 3 columns = 9 numbers total

Where you've already seen matrices in AI:

- **Neural Network Weights:** When you write `W = WeightMatrix()` and `output = W × input`, that weight matrix W transforms your input vector into an output. Weights ARE matrices.
- **Transformer Attention:** The Q, K, V projections (`Q = input × Wq`) are all matrix multiplications. Attention IS matrix math.

Dimensions: 2

1.4 — Tensor: The Generalization

Here's the big idea: a tensor is NOT some scary new concept. It's simply the natural extension of what we've been building:

Name	What it is	Dimensions	Analogy
Scalar	A single number	0D	A dot
Vector	A list of numbers	1D	A line
Matrix	A grid of numbers	2D	A table / spreadsheet
Tensor	A cube / hypercube	3D, 4D, 5D...	Stacked tables

It's just: *"What if we keep adding dimensions?"*

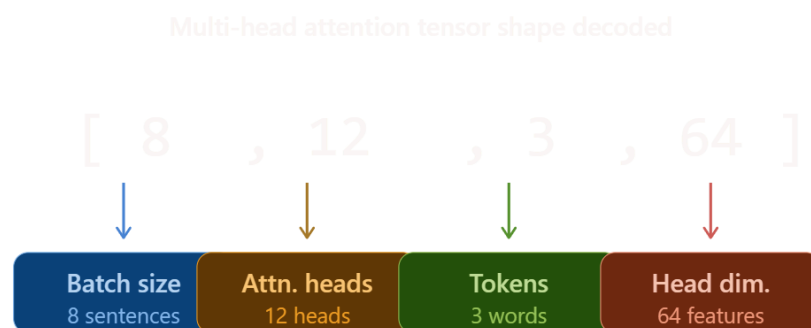
1.5 — Why Tensors Matter in Deep Learning

Real AI data isn't flat — it has layers of structure. Here's how dimensions build up when processing language:

- **A single sentence** ("The cat sat"): Each word becomes a 768-number vector. 3 words stacked = a matrix with shape `[3, 768]` — that's a **2D tensor**.
- **A batch of 8 sentences**: Stack 8 of those matrices together. Shape becomes `[8, 3, 768]` — a **3D tensor**.
- **Multi-head attention** (12 heads): Split the data across attention heads. Shape becomes `[8, 12, 3, 64]` — a **4D tensor**.

Key Insight: We need tensors because our data has structure. Sentences have words, batches have sentences, and attention has multiple heads — each layer adds a dimension.

1.6 — Shape: The Most Important Concept



Pro tip: if you understand shape, you can read any model

Shape tells you how many dimensions exist and how big each one is. It's like reading the label on a box — it tells you exactly what's inside. For the tensor shape `[8, 12, 3, 64]`, each number describes one dimension of the data.

Understanding shape is arguably the single most important debugging skill in deep learning. Most errors you'll encounter come from mismatched shapes.

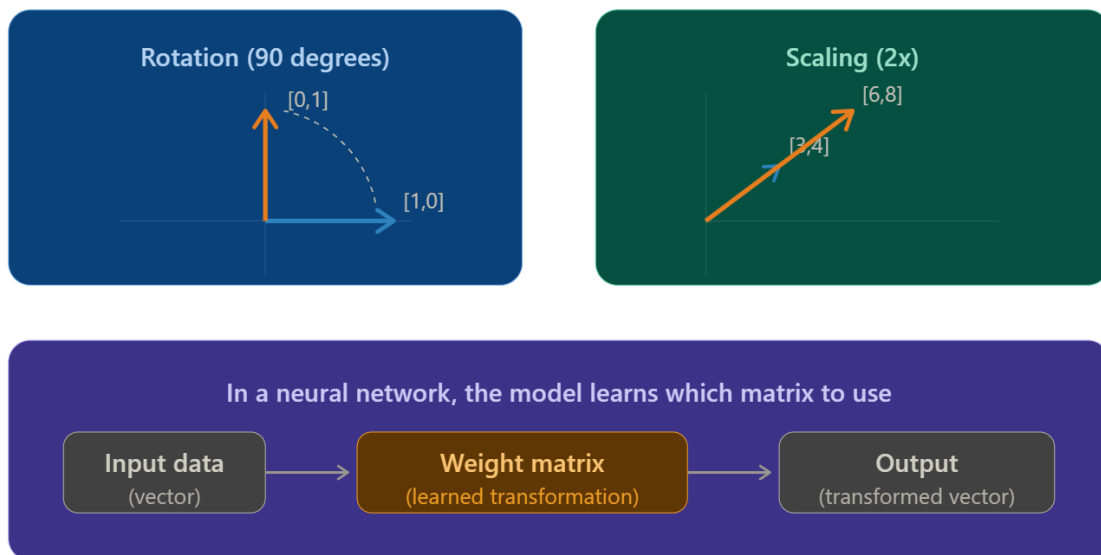
PART 2: MATRIX MAGIC — WHAT MATRICES CAN DO

This is where things get exciting. A matrix isn't just a way to store numbers — **a matrix is a TRANSFORMATION machine**. It takes a vector in and gives a different vector out. Matrices can rotate, scale, stretch, reflect, project, and transform meaning.

2.1 — Matrix × Vector = Transformation

Here's a concrete example. Start with a vector `[1, 0]` (pointing right). Multiply it by a specific matrix, and you get `[0, 1]` (pointing up). The matrix **rotated** the vector by 90 degrees!

How matrices transform vectors



Training = finding the right transformation for your data

Different matrices do different things to the same vector: one matrix might rotate it, another might scale it 2x, another might stretch it horizontally and squish it vertically. In a neural network, **training is simply the process of finding the right matrix** (the right transformation) that maps your input data to the correct output.

2.2 — Matrix Multiplication: The Core Operation

Matrix multiplication is how you combine two matrices. The key rule is: **the inner dimensions must match**.

If matrix A has shape `(3, 4)` and matrix B has shape `(4, 5)`, the inner 4s match, so the result has shape `(3, 5)`. If the inner dimensions don't match, the multiplication is impossible.

Why matrix multiplication is EVERYTHING in AI:

Operation	What it does	The math
Forward Pass	Applies learned transformations	<code>X @ W</code>
Backpropagation	Computes gradients (error signals)	<code>dL/dY @ W.T</code>
Attention	Measures similarity between tokens	<code>Q @ K.T</code>
Word Embeddings	Looks up word meaning	<code>one_hot @ W_embed</code>

Every single computation in a modern AI model is essentially a series of matrix multiplications.

2.3 — The Dot Product: The Atomic Operation

The dot product is the smallest building block of matrix multiplication. It takes two vectors and produces a single number:

`v1 = [1, 2, 3]`, `v2 = [4, 5, 6]` $v1 \cdot v2 = (1 \times 4) + (2 \times 5) + (3 \times 6) = 32$

What does this number mean? The dot product measures **similarity**. A high value means the two vectors point in the same direction (they're similar). A low or zero value means they point in different directions (they're unrelated).

This directly connects to attention in transformers! When the model computes attention for the word "sat" in the sentence "The cat sat":

- `Q_sat · K_the = 0.1` (low similarity — "sat" doesn't care much about "the")

- $Q_{\text{sat}} \cdot K_{\text{cat}} = 0.8$ (high similarity — "sat" is very related to "cat")
- $Q_{\text{sat}} \cdot K_{\text{sat}} = 0.9$ (highest — a word is most similar to itself)

Key Insight: Attention scores are just dot products. The model learns weight matrices that rotate vectors into a space where "relevant" and "similar direction" mean the same thing.

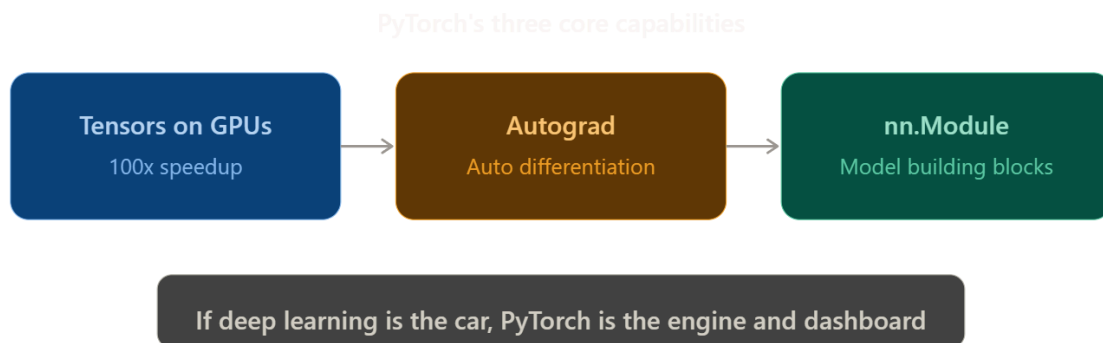
2.4 — Why GPUs Exist

Matrix multiplication is "embarrassingly parallel" — every cell in the output matrix can be calculated independently at the same time.

Feature	CPU	GPU
Cores	8-16 "complex" cores	5,000+ "simple" cores
Best for	Sequential logic, branching	Doing the same simple math thousands of times
DL speed	Baseline	Up to 100x faster

Since deep learning is basically millions of matrix multiplications, GPUs are the perfect hardware for it. That's why NVIDIA became one of the most valuable companies in the world.

PART 3: PyTorch — THE TOOL



PyTorch is the standard tool used by OpenAI, Anthropic, Google DeepMind, Tesla, and virtually every AI research lab. It gives you three superpowers: GPU-

accelerated tensors, automatic gradient computation, and high-level model building blocks.

3.1 — Creating Tensors in PyTorch

python

```
import torch

# A scalar (0D Tensor)
x = torch.tensor(5.0)

# A vector (1D Tensor)
v = torch.tensor([1.0, 2.0, 3.0])

# A matrix (2D Tensor)
m = torch.tensor([[1, 2], [3, 4]])

# A 3D tensor (e.g., Batch x Height x Width)
t = torch.zeros(2, 3, 4)    # shape [2, 3, 4]

# Check shape – you'll do this ALL the time
print(t.shape)    # torch.Size([2, 3, 4])
```

Checking `.shape` is the most common debugging operation in deep learning. Get used to printing it constantly!

3.2 — GPU Acceleration: The Magic Line

Moving a tensor from CPU to GPU is a single line of code, but it can give you up to **100x speedup** for large matrix operations:

python

```
# Create a tensor (on CPU by default)
x = torch.randn(1000, 1000)

# THE MAGIC LINE – move it to GPU
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)

# Now all operations on x happen on the GPU (5,000+ cores!)
```

That's it. One line changes where the math happens — from 8-16 CPU cores to 5,000+ GPU cores.

3.3 — Matrix Multiply with the `@` Operator

python

```
# Two matrices
A = torch.randn(3, 4) # Shape: [3, 4]
B = torch.randn(4, 5) # Shape: [4, 5]

# Matrix multiplication
C = A @ B # Shape: [3, 5]

# Or the explicit function:
C = torch.matmul(A, B)
```

The Rule: Inner dimensions MUST match! $(3, 4) @ (4, 5) \rightarrow (3, 5)$ works because both 4s match. $(3, 4) @ (5, 6)$ would crash because $4 \neq 5$.

3.4 — Reshaping Tensors

Two essential operations for rearranging data without changing the actual numbers:

.view() — Reinterpret the shape:

python

```
x = torch.randn(8, 3, 768)
y = x.view(24, 768) # Flatten batch and tokens: 8*3 = 24
# Shape: [8, 3, 768] → [24, 768]
```

.permute() — Swap axes:

python

```
x = torch.randn(8, 12, 3, 64)
y = x.permute(0, 2, 1, 3) # Swap heads and tokens
# Shape: [8, 12, 3, 64] → [8, 3, 12, 64]
```

The Rule: The total number of elements must remain exactly the same. You're just reshaping the container, not changing the data.

3.5 — Broadcasting: PyTorch's Secret Weapon

Broadcasting lets PyTorch automatically "stretch" smaller tensors to match larger ones during operations:

python

```
A = torch.ones(3, 4) # Shape: [3, 4]
B = torch.tensor([1, 2, 3, 4]) # Shape: [4]

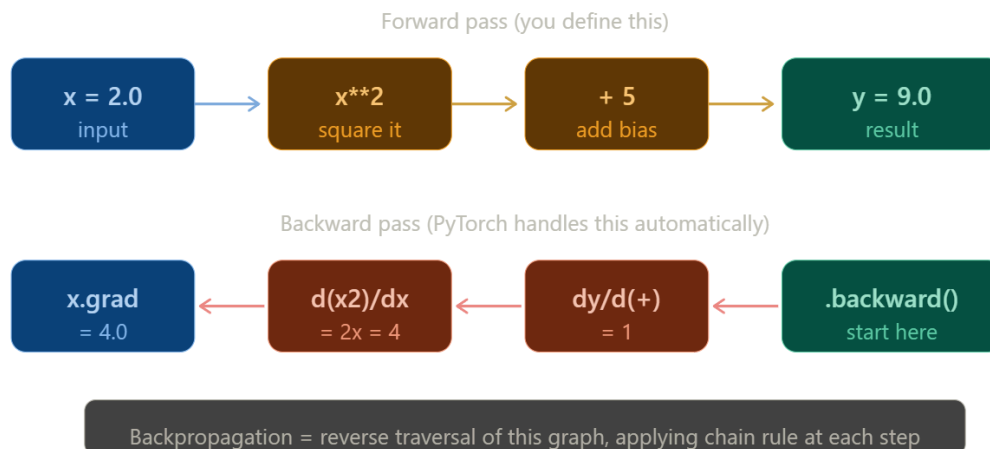
C = A + B # Result Shape: [3, 4]
# PyTorch "stretches" B and adds [1,2,3,4] to EVERY row of A
```

The Rule: Dimensions must be equal, or one of them must be 1.

3.6 — Autograd: The Big One

This is **why PyTorch exists**. Autograd computes gradients (derivatives) automatically. No more manual calculus. No more chain rule errors.

How autograd works: forward pass builds graph, backward pass computes gradients



Here's how it works in code:

python

```
import torch

# 1. Create a tensor and tell PyTorch to track it
x = torch.tensor(2.0, requires_grad=True)

# 2. Define a function: y = x^2 + 5
y = x**2 + 5          # y = 4 + 5 = 9

# 3. Compute gradients automatically!
y.backward()

# 4. The derivative: dy/dx = 2x = 2(2) = 4
print(x.grad)         # Output: tensor(4.0)
```

How it works under the hood:

- When you set `requires_grad=True`, PyTorch starts recording every operation on that tensor (like a tape recorder).
- It builds a dynamic computation graph as your code runs.
- When you call `.backward()`, it traverses this graph in reverse and applies the chain rule at every step to compute all gradients.

Why `torch.no_grad()` exists: It tells PyTorch to stop recording, which saves memory during inference (when you're using the model, not training it).

Why `optimizer.zero_grad()` exists: PyTorch accumulates gradients by default. If you call `.backward()` twice without clearing, the gradients ADD UP, which corrupts your training.

3.7 — `nn.Module` : Building Models

Every PyTorch model follows the same pattern:

python

```
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)    # Matrix: 128×784
        self.relu = nn.ReLU()                # Activation function
        self.layer2 = nn.Linear(128, 10)     # Matrix: 10×128

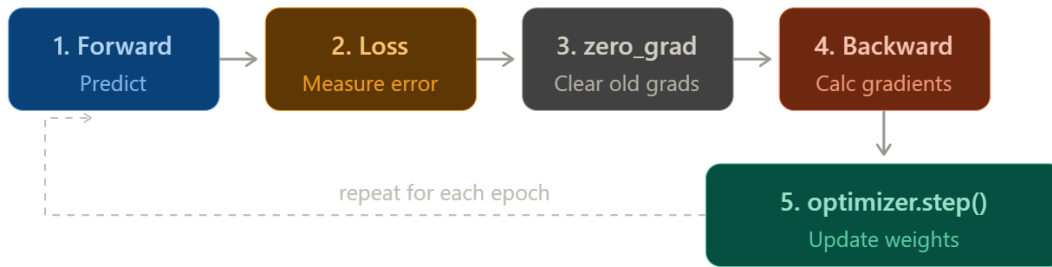
    def forward(self, x):
        x = self.layer1(x)    # Transform: 784 → 128
        x = self.relu(x)     # Filter out noise
        x = self.layer2(x)    # Transform: 128 → 10
        return x
```

That's it. You define layers in `__init__` (each layer is a matrix/transformation), and connect them in `forward` (this is how data flows through the model).

What's inside `nn.Linear(784, 128)` ? Just a 128×784 weight matrix and a bias vector. When data passes through, it computes `input @ weight.T + bias`. It's matrix multiplication and vector addition — nothing more.

3.8 — The Training Loop: The 5-Line Heartbeat

The training loop — same pattern from MNIST to GPT-4



```
for epoch in range(10):
    y_pred = model(x_batch)          # 1. Forward
    loss = criterion(y_pred, y_batch) # 2. Loss
    optimizer.zero_grad()            # 3. Clear
    loss.backward()                   # 4. Backward
    optimizer.step()                  # 5. Update
```

This 5-step pattern is the heartbeat of every deep learning model ever built. Here's what each step does in plain English:

1. **Forward** — Feed data through the model and get a prediction. "How did we do?"
2. **Loss** — Compare prediction to the correct answer. "How wrong were we?"
3. **zero_grad** — Clear old gradients so they don't pile up. (Skip this and your model explodes!)
4. **Backward** — PyTorch automatically computes how each weight contributed to the error.
5. **Step** — Nudge each weight slightly in the direction that reduces the error.

Repeat this loop thousands of times, and the model gradually learns the right transformations.

3.9 — Live Demo Recap: MNIST in Action

The lecture demonstrated training a simple neural network on handwritten digits (MNIST dataset). Here's what happened step by step:

1. **Flattening**: Each 28×28 pixel image was flattened into a 784-number vector

2. **Transform 1:** `[784] @ [784, 128] → [128]` features (the first weight matrix extracts patterns)
3. **Activation (ReLU):** Filters out noise by zeroing negative values
4. **Transform 2:** `[128] @ [128, 10] → [10]` digit scores (one score per digit 0-9)

The highest score wins — that's the model's prediction. Training found the perfect weight matrices to map pixel patterns to digit labels, achieving 98% accuracy in under 60 seconds.

SUMMARY: Everything Connects

Concept	What it is	Why it matters
Tensors	Multi-dimensional containers for data	All AI data lives in tensors
Matrices	2D grids that act as transformations	Weights, attention — everything is a matrix
Matrix Multiplication	The core computation	The engine of all deep learning
Dot Product	Multiply-and-sum two vectors	Measures similarity (powers attention)
GPU	Massively parallel processor	Makes matrix math 100x faster
PyTorch Tensors	GPU-accelerated data containers	The foundation layer
Autograd	Automatic gradient computation	No more manual calculus
nn.Module	Model building pattern	Standard way to build any model
Training Loop	The 5-line pattern	Same from MNIST to GPT-4

The one sentence summary: **Deep learning is just finding the right matrix transformations to map input data to correct outputs, and PyTorch makes that process fast, automatic, and scalable.**